```
    }
}
```

The class has a constructor that takes a `DbContextOptions` object as a parameter. You will configure this object when registering the context as a service, providing a connection string. First, you need to add a connection string to the application. You will use the main configuration file for this purpose. If you recall from chapter 2, this is the appSettings.json file. You will add a property called `ConnectionStrings`. The naming of this property, or *section*, is important because it is a convention the configuration API relies on to locate connection strings. Then you will provide a name for the connection string and its value.

**Listing 8.6 Adding a connection string to appSettings.json**

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft": "Warning",
      "Microsoft.Hosting.Lifetime": "Information"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "CityBreaksContext": "Data source=Data/CityBreaks.db"    ❶
  }
}
```

❶ The connections strings section and the actual connection string

SQLite connection strings are nice and simple, representing the path to the database file. In your application, you will house the database file in the Data folder alongside the context. The file will be created by EF Core when you run your first migration a bit later.

The final step is to register the context with the service container. As you learned from the last chapter, you do this in the Program.cs file. You need to